

M2: Crafting Reproducible Reports

Environmental Data Exploration

John Fay & Luana Lima

Learning Outcomes

By the end of this part of the module, you should be able to:

- Control document-level features using YAML.
- Control whether code, output, messages, and warnings appear in a knitted report.
- Adjust figure presentation and add informative captions.
- Use Markdown to organize and format report content.
- Present summary data as reader-friendly tables using `knitr::kable()`.
- Use inline R to keep reported values connected to the underlying analysis.
- Evaluate and refine a knitted report for its intended audience.

From Analysis to Communication

In **Video 1**, we introduced R Markdown as a reproducible document format with three main components:

1. **YAML** – document-level information and settings;
2. **Markdown text** – the narrative and formatting; and
3. **code chunks** – executable R code.

We also introduced the basic workflow of writing, running code, knitting, and reviewing the output.

Now that we have learned some R programming, we can return to R Markdown with a different goal: **turning a working analysis into a report designed for a reader.**

It is useful to distinguish between two purposes:

- An **analysis document** is optimized for the analyst. It may contain extensive code, intermediate results, diagnostics, and notes.
- A **final report** is optimized for the audience. It presents the information needed to understand the analysis and its conclusions without unnecessary clutter.

R Markdown can support both purposes from the same reproducible source document.

Crafting Reports I: Controlling Your Output

Document-level control with YAML

YAML appears at the beginning of an R Markdown document between lines containing three dashes (`---`). YAML settings apply to the document as a whole.

Common entries include the title, subtitle, author, date, and output format. Additional settings can control features such as a table of contents.

YAML exercise

Create a new R Markdown document and:

1. Set a title, author, and date.
2. Set the output format to PDF.
3. Knit the document and inspect the output.
4. Add a subtitle.
5. Add a table of contents using `toc: true` in the appropriate PDF output settings.
6. Add several Markdown headings (`#`, `##`, `###`).
7. Knit again and observe how the headings are used to build the table of contents.

The key idea is that **YAML controls properties of the entire document**. We do not manually type a table of contents; R Markdown can generate it from the document structure.

For additional options, consult the R Markdown documentation and cheat sheets available through RStudio's Help menu.

Chunk-level control

YAML controls the document as a whole, but code-chunk options allow us to control individual pieces of an analysis.

A reproducible analysis may require code that the final reader does not need to see. Chunk options let us preserve that code while controlling its presentation.

Common chunk options

Option	Purpose
<code>echo = FALSE</code>	Run the code but hide the source code; show results.
<code>include = FALSE</code>	Run the code but hide both the code and its results.
<code>eval = FALSE</code>	Display code without executing it.
<code>warning = FALSE</code>	Hide warnings in the knitted output.
<code>message = FALSE</code>	Hide messages in the knitted output.

For example, a setup chunk that loads packages and imports data may need to run for reproducibility but may not need to appear in an audience-facing report.

When deciding what to show, ask: **What does my audience need in order to understand and evaluate this analysis?**

Do not suppress unexpected warnings simply to make a report look cleaner. Investigate them first.

Controlling figures

Figure-specific chunk options help control how plots appear in a report.

Common options include:

- `fig.cap`: add an informative figure caption;
- `fig.width`, `fig.height`: control figure dimensions;
- `fig.dim`: set width and height together;
- `out.width`, `out.height`: scale the figure in the output document; and
- `fig.align`: control alignment.

A useful figure caption should tell the reader what the figure shows rather than simply say “scatterplot” or “graph.”

```
plot(mtcars$wt, mtcars$mpg,
     xlab = "Weight",
     ylab = "Miles per gallon")
```

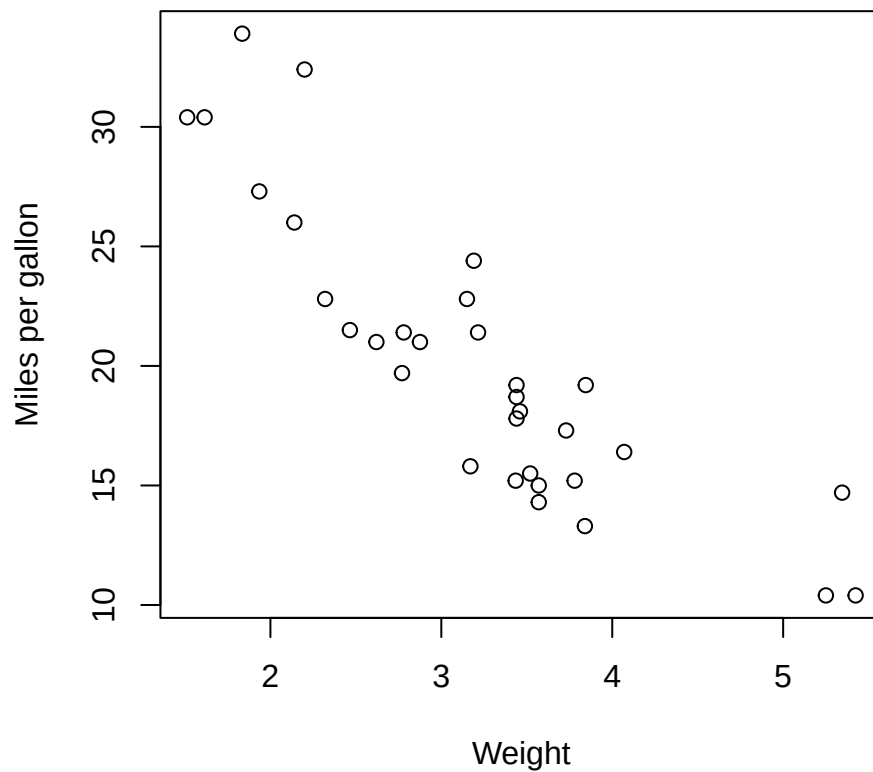


Figure 1: Example relationship between two variables.

If one code chunk generates several figures, a chunk-level caption may apply to all of them. When individual figures require distinct captions, separating them into meaningful chunks is often clearer.

Crafting Reports II: Formatting Text, Figures, and Tables

Organizing text with Markdown

Markdown formatting should help readers understand the structure of the report.

Common syntax includes:

- #, ##, ### for headings and subheadings;
- *text* for *italic* text;
- **text** for **bold** text;
- numbered lists when sequence matters;
- bulleted lists when order does not matter;
- > for block quotes; and
- Markdown syntax for links and images.

Use formatting intentionally. A report should be easy to scan without becoming visually cluttered.

Documentation and the Visual Editor

You do not need to memorize every Markdown option. Useful resources include:

- **Help > Markdown Quick Reference** in RStudio;
- the R Markdown cheat sheet; and
- the R Markdown reference documentation.

RStudio also provides a **Visual Editor**, which offers a more word-processor-like view of the document. You can switch between Source and Visual modes to see how formatting choices correspond to Markdown syntax.

Understanding the source syntax is still useful for troubleshooting and reproducibility, even if you prefer to edit visually.

Links and images

Use descriptive text for hyperlinks rather than displaying long URLs when possible.

Important images used by a report should generally be stored in an organized location within the project so that the document does not unnecessarily depend on an external website remaining available.

Presenting tables with `knitr::kable()`

A data frame that is useful inside RStudio is not automatically a presentation-ready table. The `kable()` function from the `knitr` package provides a simple way to create cleaner tables in knitted documents.

```
summary_table <- data.frame(
  Lake = c("Lake A", "Lake B", "Lake C"),
  Mean_TN = c(0.42, 0.57, 0.38),
  SD_TN = c(0.08, 0.11, 0.06)
)

knitr::kable(
```

```
summary_table,
col.names = c("Lake", "Mean Total Nitrogen", "SD Total Nitrogen"),
caption = "Summary statistics for total nitrogen by lake."
)
```

Table 2: Summary statistics for total nitrogen by lake.

Lake	Mean Total Nitrogen	SD Total Nitrogen
Lake A	0.42	0.08
Lake B	0.57	0.11
Lake C	0.38	0.06

Notice the distinction between **analysis objects** and **presentation**. Variable names that are convenient in code may be replaced with reader-friendly column labels in the report without changing the underlying object.

Also consider whether a table belongs in the report at all. A summary table is often more useful than printing hundreds of raw observations.

Crafting Reports III: From Analysis to a Polished Report

The final step is to apply these tools together rather than treating them as isolated formatting commands.

Start with an R Markdown document that runs successfully but has not yet been optimized for a reader. Knit it once and review the output before making changes.

1. Improve the document structure

Use informative headings and, when appropriate, add a table of contents. The document structure should be understandable at a glance.

2. Clean up setup code

Package loading, data import, and other setup operations may be essential to the analysis without being useful in the final report. Consider `include = FALSE` for setup chunks when appropriate.

3. Decide which analytical code to show

Some audiences benefit from seeing code; others primarily need methods, results, figures, and interpretation. Use `echo = FALSE` selectively when code should run but does not need to be displayed.

4. Handle routine messages and warnings

Routine package messages can often be hidden with `message = FALSE`. Warnings should be investigated before deciding whether suppressing them is appropriate.

5. Improve figures

Check figure size, readability, axis labels, and captions. A figure should have a clear purpose in the report and should be discussed in the surrounding text.

6. Improve tables

Use reader-friendly tables rather than raw printed data frames. Provide informative captions and meaningful column labels.

7. Add navigation when useful

Longer PDF reports may benefit from a table of contents and, when appropriate, lists of figures or tables.

For LaTeX/PDF output, commands such as the following may be useful:

List of Figures

1	Example relationship between two variables.	3
---	---	---

List of Tables

2	Summary statistics for total nitrogen by lake.	5
---	--	---

Or simply add:

List of Figures

List of Tables

These features are not necessary for every report. Use them when they improve navigation.

8. Control page breaks when needed

For PDF output, `\newpage` can be used when an intentional page break improves readability. Use manual page breaks sparingly; a little white space is usually preferable to awkwardly separating a heading, figure, table, or discussion.

9. Connect the narrative to the analysis with inline R

Avoid manually typing values into the narrative when those values are calculated by the analysis.

For example, suppose an object called `cars` contains the dataset. Instead of manually typing the number of observations, you could write an inline expression such as:

The raw dataset contains 50 observations.

When the data change, the value in the sentence can update automatically the next time the report is knitted.

Inline R is useful for sample sizes, means, percentages, dates, model results, and other values that come directly from the analysis.

10. Knit and review as a reader

A successful knit means that the document rendered; it does **not** necessarily mean that the report communicates effectively.

After knitting, review the report from the perspective of the intended audience:

- Does the organization make sense?
- Are headings informative?
- Are figures and tables readable and discussed in the text?
- Are captions descriptive?
- Is unnecessary code distracting from the results?
- Are there messages or warnings that should be investigated or removed?
- Are page breaks awkward?
- Are values in the narrative connected to the analysis where practical?
- Can the reader understand what was done, what was found, and why it matters?

Practice: Report Makeover

Use a working R Markdown analysis and improve it step by step.

1. Knit the document **before** making changes and inspect the output.
2. Add or improve the table of contents.
3. Configure the setup chunk so that required code runs without cluttering the final report.
4. Decide which analytical code should remain visible to the reader.
5. Remove routine messages and investigate any warnings.
6. Improve figure dimensions and add informative captions.
7. Present at least one summary data frame with `knitr::kable()`.
8. Add a table caption and reader-friendly column names.
9. Add navigation such as a list of figures or tables if the report is long enough to benefit from it.
10. Replace at least one manually typed analytical result with inline R.
11. Knit again and compare the revised report with the original.

Part 2 Wrap-Up

Crafting a report is not simply a matter of making output look attractive. It is the process of deciding how to communicate a reproducible analysis effectively.

The `.Rmd` source should preserve the analytical workflow. The knitted report should present the information the intended audience needs.

Throughout **Environmental Data Exploration**, continue using the same cycle:

organize the project -> write the analysis -> run the code -> interpret the results -> communicate the results reproducibly.

Optional: Understanding knitting errors

You do not need to understand the full rendering pipeline to use R Markdown successfully. If you encounter knitting problems, however, it can help to know that rendering involves multiple steps and tools. Treat this as troubleshooting material rather than a core concept for this module.

Useful first checks include:

- Read the error shown in the Render/Knit output.
- Confirm that the document runs from a clean R session.
- Check that required files use correct relative paths.
- Make sure required packages are installed and loaded.
- For PDF output, check that the required LaTeX/TinyTeX components are available.